AI-Powered Blood Donor Matching & Emergency Request Management System

Software Engineering & Project Management System Proposal

Student Name: Saiba Khan

Student ID: IT-24006

Course Name: Software Engineering and Project Management

1. Executive Summary & Problem Statement

1.1 Background

Medical emergencies requiring urgent blood transfusions frequently encounter critical delays due to inefficient donor identification and manual communication channels. Traditional blood bank systems and social media request groups lack real-time proximity tracking, algorithmic matching, and automated response verification, leading to prolonged search times during critical emergency windows.

1.2 Problem Statement

In life-threatening situations, matching patients with suitable blood donors depends on multi-variable criteria including exact blood compatibility, geographic location, real-time donor availability, recent donation eligibility history (at least 90 days interval), and donor reliability. Manual processing fails to rapidly evaluate these multi-factor constraints simultaneously, often contacting unavailable or ineligible donors first.

1.3 Proposed Solution

The AI-Powered Blood Donor Matching and Emergency Request Management System introduces an automated web application that leverages a multi-criteria decision algorithm to evaluate registered donors instantly upon emergency creation. The system calculates dynamic compatibility scores (0-100%) and prioritizes eligible candidates within immediate geographic proximity.

2. Project Objectives & Functional Scope

2.1 Primary Objectives

- Reduce critical donor discovery time from hours to seconds through automated multi-factor scoring algorithms.
- Optimize donor outreach by filtering candidates based on blood group compatibility matrix, 90-day eligibility rules, and GPS/district proximity metrics.
- Provide real-time dashboards for Donors, Patients/Hospitals, and System Administrators.
- Implement an interactive notification and response mechanism to monitor donor reliability and response history.

2.2 Functional Modules

- **Donor Management Module:** Handles registration, medical history profile, active availability toggles, and donation history tracking.
- **Emergency Request Module:** Allows creation, editing, and closing of critical patient requests with hospital locations and required urgency levels.
- **AI Matching Engine:** Computes multi-variable compatibility scores combining Blood Matrix Match (40%), Distance Proximity (30%), Eligibility Criteria (15%), and Reliability Index (15%).
- **Donor Dashboard:** Enables donors to toggle real-time status, view localized requests, and accept/reject dispatch alerts.
- **Admin Management Dashboard:** Provides platform monitoring, user verification, system analytics, and audit logging.

3. Methodology & Software Process Model

This project adopts the Agile Scrum Framework due to its iterative nature, enabling continuous testing, modular verification, and rapid feedback loops required when refining ranking algorithms and interactive interfaces.

3.1 Process Lifecycle Sprints

- **Sprint 0 (Requirements & Architecture):** Domain modeling, UML schema design, and technical stack selection.
- **Sprint 1 (Core Engine & DB):** Database schema implementation, JPA entities, and core heuristic algorithm development.
- **Sprint 2 (API & Module Logic):** RESTful API implementation for Donor and Request modules.
- **Sprint 3 (Frontend Integration):** Dashboard, notification UI, and algorithmic ranking inspector visualization.
- **Sprint 4 (Testing & Deployment):** Unit testing, integration testing, algorithmic optimization, and system deployment.

4. Completion Schedule, Milestones & Deliverables

4.1 Project Milestones Table

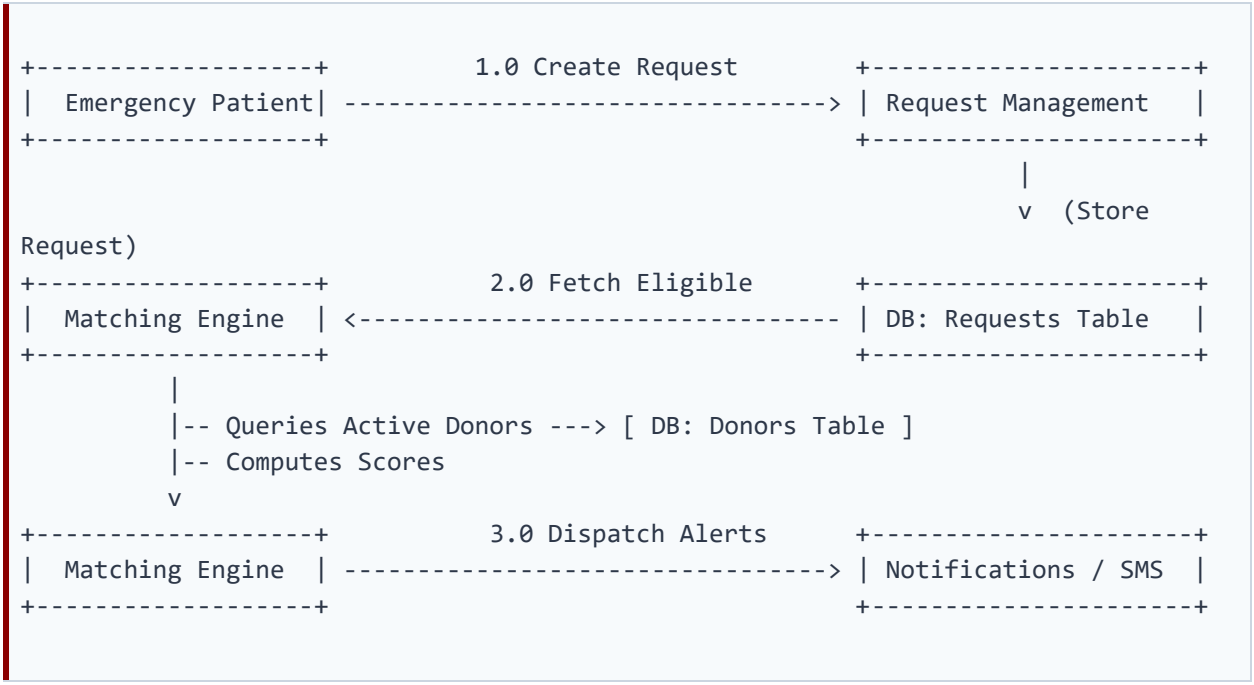
Milestone	Deliverable Name	Key Output / Content	Deadline
M1	Project Proposal & Architecture	Requirements, UML Diagrams, Tech Stack	Week 2
M2	Backend Engine & Schema	MySQL Database Schema, Spring Data Entities	Week 5
M3	AI Matching Engine Core	Weighted scoring service & REST API	Week 8
M4	Frontend & Dashboard Integration	Single-Page Application (SPA) / UI Interfaces	Week 11
M5	System Validation & Testing	End-to-End Execution, Bug Fixes	Week 13
M6	Final Release & Presentation	Source Code Repository & Final Report	Week 14

4.2 Project Schedule (14-Week Timeline)

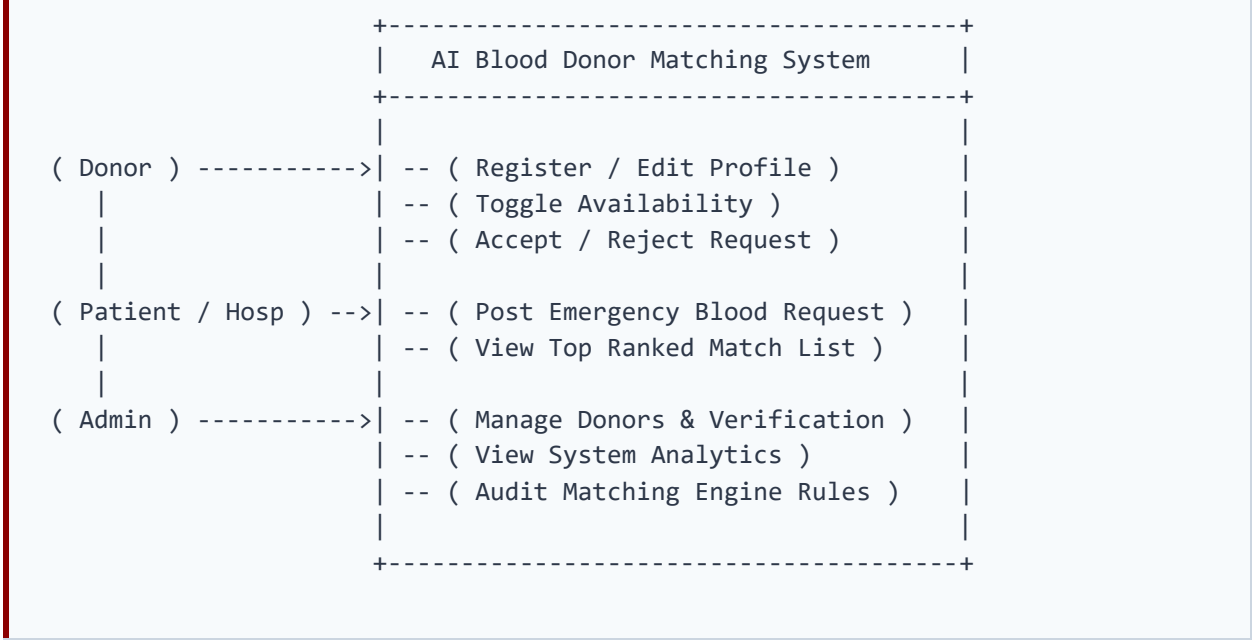
```
+-----------------------------------+---------------------------------------+
| Task / Milestone                  | Timeline (Weeks)                      |
+-----------------------------------+---------------------------------------+
| Requirements & UML Diagrams       | [W1 - W2]  --> (M1 Delivered)         |
| DB Schema & Entity Modeling       | [W3 - W5]  --> (M2 Delivered)         |
| AI Engine & Logic Development     | [W5 - W8]  --> (M3 Delivered)         |
| UI & Dashboard Development        | [W8 - W11] --> (M4 Delivered)         |
| System Integration & Testing      | [W11 - W13]-> (M5 Delivered)          |
| Final Release & Handover          | [W13 - W14]-> (M6 Complete)           |
+-----------------------------------+---------------------------------------+
```

5. Unified Modeling Language (UML) Diagrams

5.1 Level 1 Data Flow Diagram (DFD)



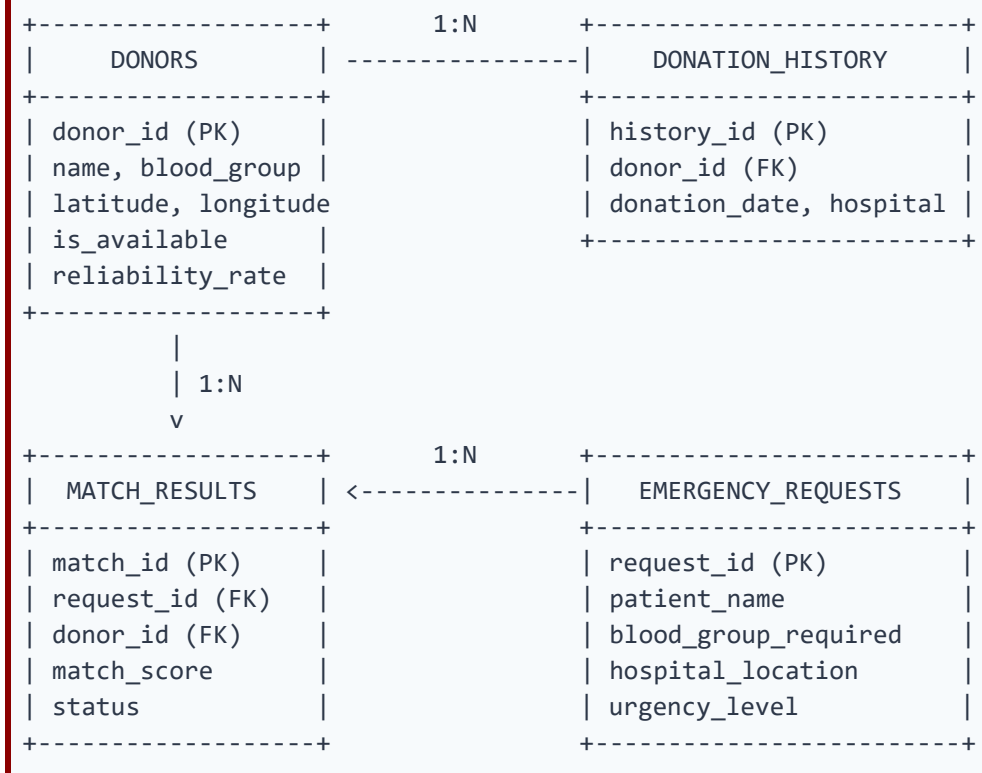
5.2 Use Case Diagram



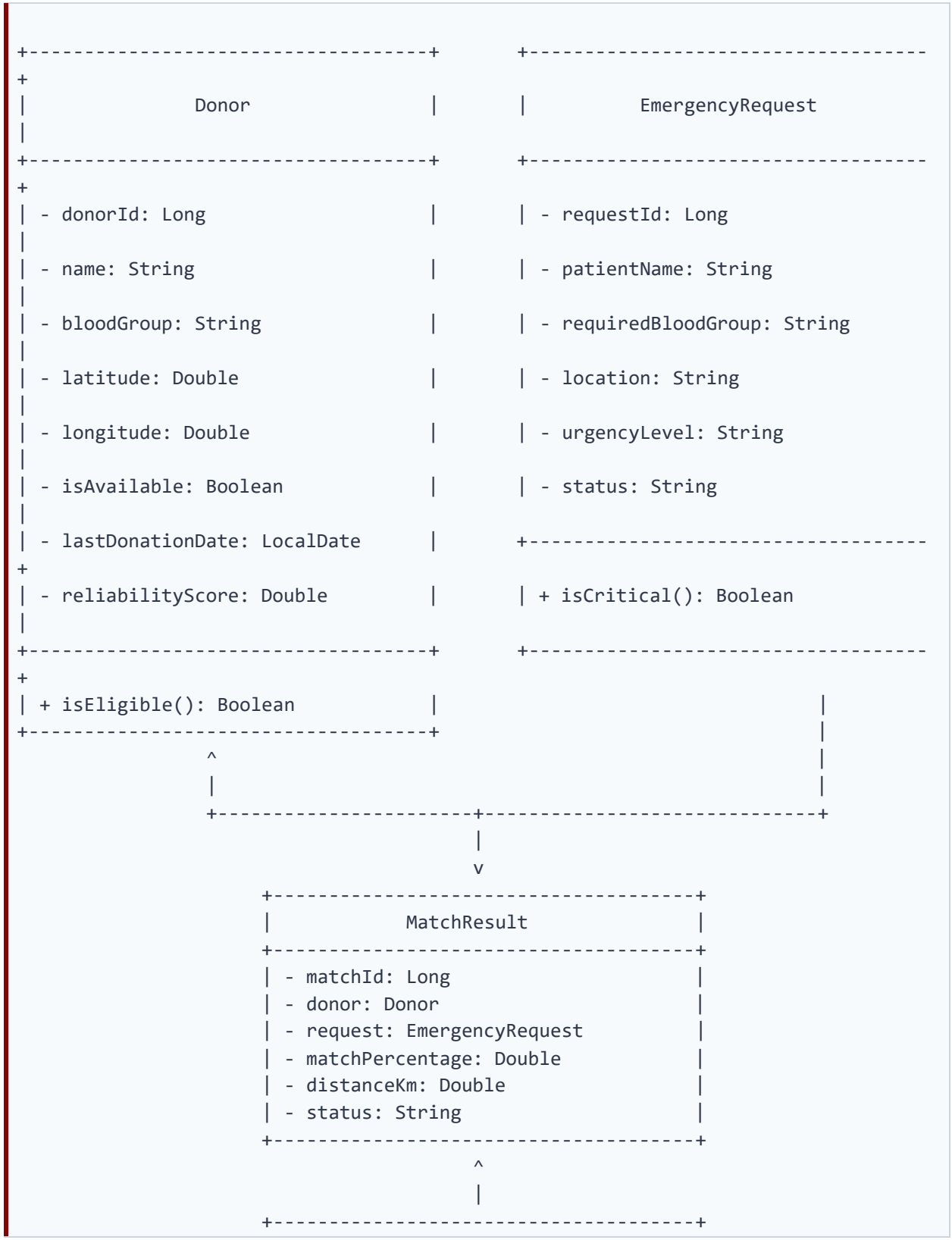
5.3 Activity Diagram

```
(*) -> [Patient Posts Emergency Request]
    -> [Validate Required Blood Group & Location]
    -> [Query Database for Active & Eligible Donors]
    -> [For Each Donor: Compute Multi-Factor Score]
           |-- Blood Compatibility Weight (40%)
           |-- Proximity Distance Penalty (30%)
           |-- 90-Day Eligibility Criteria (15%)
           |-- Historical Response Reliability (15%)
    -> [Rank Donors in Descending Order of Match %]
    -> [Display Top Matches on Patient Interface]
    -> [Send Dispatch Alert Notification to Donor]
    -> (*)
```

5.4 Entity Relationship Diagram (ERD)



5.5 Class Diagram



```
                    |          MatchingEngineService       |
                    +--------------------------------------+
                    | + calculateMatchScore() : Double     |
                    | + findTopMatches() : List<MatchResult|
                    | - evaluateBloodCompatibility()       |
                    | - evaluateDistancePenalty()          |
                    +--------------------------------------+
```

5.6 Object Diagram

```
+--------------------------------------------------+
| req101 : EmergencyRequest                        |
+--------------------------------------------------+
| requestId = 101                                  |
| patientName = "Rafiqul Islam"                    |
| requiredBloodGroup = "O+"                        |
| location = "Tangail General Hospital"            |
| urgencyLevel = "CRITICAL"                        |
+--------------------------------------------------+
                         |
                         v
+--------------------------------------------------+
| match801 : MatchResult                           |
+--------------------------------------------------+
| matchId = 801                                    |
| matchPercentage = 96.0                           |
| distanceKm = 3.2                                 |
| status = "RECOMMENDED"                           |
+--------------------------------------------------+
                         |
                         v
+--------------------------------------------------+
| donor201 : Donor                                 |
+--------------------------------------------------+
| donorId = 201                                    |
| name = "Tanvir Ahmed"                            |
| bloodGroup = "O+"                                |
| isAvailable = true                               |
| lastDonationDate = 2026-05-10                    |
| reliabilityScore = 98.5                          |
+--------------------------------------------------+
```

6. Implementation Architecture & Java Class Definitions

Below are the 5 core domain entities and services implementing the UML Class Model for the Java Spring Boot backend.

6.1 Donor Domain Entity (Donor.java)

```
package com.blooddonor.model;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Donor {
    private Long donorId;
    private String name;
    private String bloodGroup;
    private Double latitude;
    private Double longitude;
    private Boolean isAvailable;
    private LocalDate lastDonationDate;
    private Double reliabilityScore;

    public Donor() {}

    public Donor(Long donorId, String name, String bloodGroup, Double latitude,
                 Double longitude, Boolean isAvailable, LocalDate
lastDonationDate,
                 Double reliabilityScore) {
        this.donorId = donorId;
        this.name = name;
        this.bloodGroup = bloodGroup;
        this.latitude = latitude;
        this.longitude = longitude;
        this.isAvailable = isAvailable;
        this.lastDonationDate = lastDonationDate;
        this.reliabilityScore = reliabilityScore;
    }

    public Boolean isEligible() {
        if (lastDonationDate == null) return true;
        long daysBetween = ChronoUnit.DAYS.between(lastDonationDate,
LocalDate.now());
        return daysBetween >= 90;
    }

    // Getters and Setters
    public Long getDonorId() { return donorId; }
```

```
    public void setDonorId(Long donorId) { this.donorId = donorId; }
    public String getName() { return name; }
    public String getBloodGroup() { return bloodGroup; }
    public Double getLatitude() { return latitude; }
    public Double getLongitude() { return longitude; }
    public Boolean getIsAvailable() { return isAvailable; }
    public void setIsAvailable(Boolean isAvailable) { this.isAvailable =
isAvailable; }
    public LocalDate getLastDonationDate() { return lastDonationDate; }
    public Double getReliabilityScore() { return reliabilityScore; }
}
```

6.2 Emergency Request Domain Entity (EmergencyRequest.java)

```
package com.blooddonor.model;

public class EmergencyRequest {
    private Long requestId;
    private String patientName;
    private String requiredBloodGroup;
    private String location;
    private Double latitude;
    private Double longitude;
    private String urgencyLevel; // LOW, MEDIUM, CRITICAL

    public EmergencyRequest(Long requestId, String patientName, String
requiredBloodGroup,
                            String location, Double latitude, Double longitude,
String urgencyLevel) {
        this.requestId = requestId;
        this.patientName = patientName;
        this.requiredBloodGroup = requiredBloodGroup;
        this.location = location;
        this.latitude = latitude;
        this.longitude = longitude;
        this.urgencyLevel = urgencyLevel;
    }

    public Boolean isCritical() {
        return "CRITICAL".equalsIgnoreCase(this.urgencyLevel);
    }

    // Getters and Setters
    public Long getRequestId() { return requestId; }
    public String getPatientName() { return patientName; }
    public String getRequiredBloodGroup() { return requiredBloodGroup; }
```

```
    public Double getLatitude() { return latitude; }
    public Double getLongitude() { return longitude; }
    public String getUrgencyLevel() { return urgencyLevel; }
}
```

6.3 Match Result Entity (MatchResult.java)

```
package com.blooddonor.model;

public class MatchResult implements Comparable<MatchResult> {
    private Long matchId;
    private Donor donor;
    private EmergencyRequest request;
    private Double matchPercentage;
    private Double distanceKm;
    private String status;

    public MatchResult(Long matchId, Donor donor, EmergencyRequest request,
                       Double matchPercentage, Double distanceKm, String status) {
        this.matchId = matchId;
        this.donor = donor;
        this.request = request;
        this.matchPercentage = matchPercentage;
        this.distanceKm = distanceKm;
        this.status = status;
    }

    @Override
    public int compareTo(MatchResult other) {
        return Double.compare(other.matchPercentage, this.matchPercentage);
    }

    // Getters
    public Donor getDonor() { return donor; }
    public Double getMatchPercentage() { return matchPercentage; }
    public Double getDistanceKm() { return distanceKm; }
}
```

6.4 AI Matching Engine Service (MatchingEngineService.java)

```
package com.blooddonor.service;

import com.blooddonor.model.Donor;
import com.blooddonor.model.EmergencyRequest;
```

```
import com.blooddonor.model.MatchResult;

import java.util.*;

public class MatchingEngineService {

    public Double calculateMatchScore(Donor donor, EmergencyRequest request,
Double distanceKm) {
        if (!donor.getIsAvailable() || !donor.isEligible()) {
            return 0.0;
        }

        // 1. Blood Compatibility Matrix (40 Points Max)
        double bloodScore = evaluateBloodCompatibility(donor.getBloodGroup(),
request.getRequiredBloodGroup());

        // 2. Distance Score (30 Points Max)
        double distanceScore = Math.max(0, 30.0 - (distanceKm * 1.5));

        // 3. Eligibility/Recency Bonus (15 Points Max)
        double eligibilityScore = 15.0;

        // 4. Historical Reliability (15 Points Max)
        double reliabilityScore = (donor.getReliabilityScore() / 100.0) * 15.0;

        double totalScore = bloodScore + distanceScore + eligibilityScore +
reliabilityScore;
        return Math.min(100.0, totalScore);
    }

    private double evaluateBloodCompatibility(String donorType, String
recipientType) {
        if (donorType.equals(recipientType)) return 40.0;
        if (donorType.equals("O-")) return 38.0; // Universal Donor
        if (donorType.equals("O+") && recipientType.contains("+")) return 35.0;
        return 0.0;
    }

    public List<MatchResult> findTopMatches(List<Donor> donors, EmergencyRequest
request) {
        List<MatchResult> results = new ArrayList<>();
        long idCounter = 1;

        for (Donor donor : donors) {
            double distance = computeHaversineDistance(
                donor.getLatitude(), donor.getLongitude(),
                request.getLatitude(), request.getLongitude()
            );
```

```
            double score = calculateMatchScore(donor, request, distance);
            if (score > 0) {
                results.add(new MatchResult(idCounter++, donor, request, score,
distance, "RECOMMENDED"));
            }
        }

        Collections.sort(results);
        return results;
    }

    private double computeHaversineDistance(double lat1, double lon1, double lat2,
double lon2) {
        double r = 6371; // Earth's Radius in KM
        double dLat = Math.toRadians(lat2 - lat1);
        double dLon = Math.toRadians(lon2 - lon1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
                   Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
*
                   Math.sin(dLon / 2) * Math.sin(dLon / 2);
        return r * (2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a)));
    }
}
```

6.5 Main Application Execution Test (MainApplication.java)

```
package com.blooddonor;

import com.blooddonor.model.*;
import com.blooddonor.service.MatchingEngineService;

import java.time.LocalDate;
import java.util.*;

public class MainApplication {
    public static void main(String[] args) {
        System.out.println("--- Starting AI Blood Donor Matching System ---");

        // 1. Create Emergency Request in Tangail
        EmergencyRequest request = new EmergencyRequest(
            1L, "Patient X", "O+", "Tangail Sadar Hospital", 24.2513, 89.9167,
"CRITICAL"
        );

        // 2. Prepare Sample Registered Donors
```

```
        List<Donor> donors = Arrays.asList(
            new Donor(101L, "Tanvir Ahmed", "O+", 24.2600, 89.9300, true,
LocalDate.now().minusDays(120), 95.0),
            new Donor(102L, "Kamrul Hassan", "O-", 24.2800, 89.9500, true,
LocalDate.now().minusDays(100), 90.0),
            new Donor(103L, "Sabrina Khan", "A+", 24.2500, 89.9100, true,
LocalDate.now().minusDays(200), 88.0)
        );

        // 3. Execute Matching Engine
        MatchingEngineService engine = new MatchingEngineService();
        List<MatchResult> matches = engine.findTopMatches(donors, request);

        // 4. Print Ranked Results
        System.out.println("\nRanked Donor Candidates for Patient: " +
request.getPatientName());
        for (MatchResult res : matches) {
            System.out.printf("Donor: %-15s | Match: %6.2f%% | Distance: %4.2f
km%n",
                res.getDonor().getName(), res.getMatchPercentage(),
res.getDistanceKm());
        }
    }
}
```

7. Project Repository & Submission Summary

This project proposal provides a complete blueprint for the AI-Powered Blood Donor Matching and Emergency Request Management System designed for the course Software Engineering and Project Management.

GitHub Repository Link: https://github.com/saibakhan1104-png/AI-Powered-Blood-Donor-Matching-Emergency-Request-Management-System